

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11 - ERROR HANDLING**

Disusun oleh:

Paulus Roberto Apolos Siagian
NIM: 105224020

**LABORATORIUM KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN ILMU KOMPUTER
UNIVERSITAS PERTAMINA
2026**

I. TUJUAN PRAKTIKUM

Setelah menyelesaikan praktikum modul ini, mahasiswa diharapkan mampu:

- Memahami konsep exception dan exception handling dalam Java.
- Menggunakan blok try-catch-finally untuk menangani error pada program.
- Membuat custom exception yang sesuai dengan kebutuhan domain aplikasi.
- Menggunakan keyword throw dan throws secara tepat dalam program Java.
- Menerapkan prinsip best practices dalam penulisan exception handling.

II. DASAR TEORI

2.1 Pengertian Exception

Exception adalah kejadian tak terduga yang terjadi saat program dijalankan dan mengganggu alur normal eksekusi. Ketika exception terjadi, Java membuat sebuah objek exception yang berisi informasi tentang error tersebut, lalu menyerahkannya ke runtime system. Jika tidak ditangani, program akan berhenti secara tiba-tiba (crash).

2.2 Blok try-catch-finally

Blok try digunakan untuk membungkus kode yang berpotensi menimbulkan exception. Blok catch bertugas menangkap dan menangani exception yang dilempar oleh blok try. Blok finally adalah blok kode yang selalu dieksekusi, tidak peduli apakah exception terjadi atau tidak, bahkan jika blok try atau catch memiliki pernyataan return. Blok ini sangat berguna untuk menutup resource seperti koneksi database, file, atau Scanner.

2.3 Custom Exception

Custom exception dibuat untuk mendeskripsikan error yang bersifat domain-spesifik dalam aplikasi yang dibangun. Dengan membuat custom exception, kode menjadi lebih ekspresif dan mudah dibaca. Custom exception dibuat dengan meng-extends class Exception (untuk Checked Exception) atau RuntimeException (untuk Unchecked Exception).

2.4 Keyword throw dan throws

Keyword throw digunakan untuk melempar exception secara manual di dalam body method, sedangkan keyword throws ditulis pada signature method untuk mendeklarasikan bahwa method tersebut berpotensi melempar exception tertentu. Keyword throws merupakan cara mendelegasikan tanggung jawab penanganan exception kepada pemanggil method.

III. ALAT DAN BAHAN

- Komputer/Laptop dengan sistem operasi Windows/Linux/macOS
- Java Development Kit (JDK) versi 8 atau lebih baru
- IDE IntelliJ IDEA / Eclipse / Visual Studio Code

- Modul Praktikum Pemrograman Berorientasi Objek Modul 11

IV. LANGKAH PERCOBAAN

Pada praktikum ini, dibuat sebuah program Sistem Reservasi Tiket Kereta bernama "Java Express" yang mengimplementasikan konsep error handling. Program terdiri dari beberapa class yang saling bekerja sama, yaitu class custom exception, class entitas Kereta, class controller ReservasiController, dan class Main sebagai titik masuk program.

4.1 Membuat Custom Exception: DataPenumpangTidakValidException

Langkah pertama adalah membuat custom unchecked exception dengan nama DataPenumpangTidakValidException yang meng-extends RuntimeException. Exception ini digunakan untuk menangani kondisi ketika data penumpang yang dimasukkan tidak valid, seperti format NIK yang salah.

```
1 public class DataPenumpangTidakValidException extends RuntimeException{
2     public DataPenumpangTidakValidException(String message){
3         super(message);
4     }
5 }
```

Gambar 1. Class DataPenumpangTidakValidException

4.2 Membuat Custom Exception: RuteTidakDitemukanException

Selanjutnya dibuat custom checked exception RuteTidakDitemukanException yang meng-extends Exception. Exception ini digunakan untuk menangani kondisi ketika kode kereta yang dimasukkan tidak ditemukan dalam sistem.

```
1 public class RuteTidakDitemukanException extends Exception{
2     public RuteTidakDitemukanException(String message){
3         super(message);
4     }
5 }
```

Gambar 2. Class RuteTidakDitemukanException

4.3 Membuat Class Kereta

Class Kereta adalah class entitas yang merepresentasikan data kereta api dalam sistem. Class ini memiliki atribut kodeKereta, namaKereta, rute, dan sisaKursi, beserta constructor dan method getter serta method kurangiKursi() untuk mengurangi jumlah kursi tersedia setelah pemesanan.

```

1  public class Kereta{
2      private String kodeKereta;
3      private String namaKereta;
4      private String rute;
5      private int sisaKursi;
6
7      public Kereta(String kodeKereta, String namaKereta, String rute, int sisaKursi){
8          this.kodeKereta = kodeKereta;
9          this.namaKereta = namaKereta;
10         this.rute = rute;
11         this.sisaKursi = sisaKursi;
12     }
13     public String getKodeKereta(){
14         return kodeKereta;
15     }
16     public String getNamaKereta(){
17         return namaKereta;
18     }
19     public String getRute(){
20         return rute;
21     }
22     public int getSisaKursi(){
23         return sisaKursi;
24     }
25     public void kurangiKursi(int jumlah){
26         this.sisaKursi -= jumlah;
27     }
28 }

```

Gambar 3. Class Kereta

4.4 Membuat Custom Exception: TiketHabisException

Class TiketHabisException adalah custom checked exception yang meng-extends Exception. Exception ini menyimpan informasi tambahan berupa nama kereta dan sisa kursi, sehingga pengguna dapat mengetahui detail ketersediaan kursi saat exception terjadi.

```

1  public class TiketHabisException extends Exception{
2      private String namaKereta;
3      private int sisaKursi;
4
5      public TiketHabisException(String namaKereta, int sisaKursi){
6          super("Tiket gagal dipesan! Kursi untuk kereta " + namaKereta + " tidak mencukupi.");
7          this.namaKereta = namaKereta;
8          this.sisaKursi = sisaKursi;
9      }
10     public String getNamaKereta(){
11         return namaKereta;
12     }
13     public int getSisaKursi(){
14         return sisaKursi;
15     }
16 }

```

Gambar 4. Class TiketHabisException

4.5 Membuat Class ReservasiController

ReservasiController adalah class yang bertugas mengelola logika bisnis reservasi tiket. Class ini menginisialisasi daftar kereta, lalu method pesanTiket() memvalidasi input NIK, mencari kereta berdasarkan kode, memeriksa ketersediaan kursi, dan melempar

exception yang sesuai menggunakan keyword throw. Method ini juga mendeklarasikan throws untuk exception yang bersifat checked.

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class ReservasiController{
5     private List<Kereta> daftarKereta;
6
7     public ReservasiController(){
8         daftarKereta = new ArrayList<>();
9         initData();
10    }
11    private void initData(){
12        daftarKereta.add(new Kereta(kodeKereta: "K01", namaKereta: "Ango Bromo", rute: "JKT - SBY", sisaKursi: 50));
13        daftarKereta.add(new Kereta(kodeKereta: "K02", namaKereta: "Parahyangan", rute: "JKT - BDG", sisaKursi: 15));
14    }
15    public List<Kereta> getDaftarKereta(){
16        return daftarKereta;
17    }
18    public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)
19        throws RuteTidakDitemukanException, TiketHabisException{
20
21        if (nik.length() != 16 || !nik.matches("[0-9]+")){
22            throw new DataPenumpangTidakValidException(message: "Format NIK tidak valid! Harus tepat 16 digit angka.");
23        }
24        Kereta targetKereta = null;
25        for (Kereta k : daftarKereta){
26            if (k.getKodeKereta().equalsIgnoreCase(kodeKereta)){
27                targetKereta = k;
28                break;
29            }
30        }
31        if (targetKereta == null){
32            throw new RuteTidakDitemukanException("Kode kereta " + kodeKereta + " tidak ditemukan di sistem.");
33        }
34        if (jumlahTiket > targetKereta.getSisaKursi()){
35            throw new TiketHabisException(targetKereta.getNamaKereta(), targetKereta.getSisaKursi());
36        }
37
38        targetKereta.kurangiKursi(jumlahTiket);
39        System.out.println("🎫 RESERVASI TIKET BERHASIL!");
40        System.out.println("Nama Penumpang : " + namaPenumpang + " (NIK: " + nik + ")");
41        System.out.println("Kereta Api      : " + targetKereta.getNamaKereta() + " (" + targetKereta.getRute() + ")");
42        System.out.println("Jumlah Bangku  : " + jumlahTiket + " kursi");
43    }
44 }
```

Gambar 5. Class ReservasiController

4.6 Membuat Class Main

Class Main adalah titik masuk program yang mengimplementasikan menu interaktif menggunakan do-while loop. Di dalam loop, digunakan blok try-catch-finally dengan multiple catch blocks untuk menangani berbagai jenis exception secara terpisah dan informatif. Blok finally digunakan untuk menutup Scanner saat pengguna memilih keluar dari program.

```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class Main{
5     Run | Debug
6     public static void main(String[] args){
7         ReservasiController controller = new ReservasiController();
8         Scanner scanner = new Scanner(System.in);
9         int menu = 0;
10
11         System.out.println(" SELAMAT DATANG DI SISTEM JAVA EXPRESS ");
12
13         do {
14             try {
15                 System.out.println("\n=== MENU UTAMA ===");
16                 System.out.println("1. Lihat Jadwal & Sisa Kursi");
17                 System.out.println("2. Pesan Tiket Kereta");
18                 System.out.println("3. Keluar Aplikasi");
19                 System.out.print("Pilih menu (1-3): ");
20
21                 menu = scanner.nextInt();
22                 scanner.nextLine();
23
24                 switch (menu){
25                     case 1:
26                         System.out.println("\n--- JADWAL KERETA JAVA EXPRESS ---");
27                         System.out.printf("%-10s %-20s %-15s %-10s\n", "KODE", "NAMA KERETA", "RUTE", "SISA KURSI");
28                         for (Kereta k : controller.getDaftarKereta()) {
29                             System.out.printf("%-10s %-20s %-15s %-10d\n",
30                                 k.getKodeKereta(), k.getNamaKereta(), k.getRute(), k.getSisaKursi());
31                         }
32                         break;
33                     case 2:
34                         System.out.println("\n--- FORMULIR RESERVASI TIKET ---");
35                         System.out.print("Masukkan Kode Kereta (K01/K02): ");
36                         String kode = scanner.nextLine().trim();
37
38                         System.out.print("Masukkan NIK Penumpang      : ");
39                         String nik = scanner.nextLine().trim();
40
41                         System.out.print("Masukkan Nama Penumpang    : ");
42                         String nama = scanner.nextLine().trim();
43
44                         System.out.print("Masukkan Jumlah Tiket      : ");
45                         int jumlahTiket = scanner.nextInt();
46                         scanner.nextLine();
47
48                         controller.pesanTiket(kode, nik, nama, jumlahTiket);
49                         break;
50                     case 3:
51                         break;
52                     default:
53                         System.out.println(" Pilihan menu tidak valid! Sila pilih angka 1 hingga 3.");
54                 }
55
56             } catch (InputMismatchException e) {
57                 System.out.println(" ERROR INPUT: Harap masukkan angka saja!");
58                 scanner.nextLine();
59             } catch (DataPenumpangTidakValidException e) {
60                 System.out.println(" MASALAH DATA: " + e.getMessage());
61             } catch (RuteTidakDitemukanException e) {
62                 System.out.println(" MASALAH RUTE: " + e.getMessage());
63             } catch (TiketHabisException e) {
64                 System.out.println(" MASALAH KETERSEDIAAN: " + e.getMessage());
65             } catch (Exception e) {
66                 System.out.println("Detail Sisa -> Kereta: " + e.getNamaKereta() + " | Kursi Tersedia: " + e.getSisaKursi());
67             } catch (Exception e) {
68                 System.out.println(" Terjadi gangguan pada sistem: " + e.getMessage());
69             } finally {
70                 if (menu == 3){
71                     System.out.println("Terima kasih telah menggunakan sistem JAVA EXPRESS!");
72                     System.out.println("Sistem dimatikan secara aman. Sampai jumpa!");
73                     scanner.close();
74                 }
75             }
76         } while (menu != 3);
77     }
78 }
79

```

Gambar 6. Class Main

V. HASIL DAN PEMBAHASAN

5.1 Analisis Struktur Program

Program Sistem Reservasi Tiket Kereta "Java Express" yang dibuat terdiri dari enam class yang masing-masing memiliki tanggung jawab berbeda. Berikut adalah ringkasan struktur class dalam program:

Nama Class	Tipe	Jenis Exception	Fungsi
DataPenumpangTidakValidException	Unchecked	extends RuntimeException	Validasi format NIK penumpang
RuteTidakDitemukanException	Checked	extends Exception	Kode kereta tidak ditemukan
TiketHabisException	Checked	extends Exception	Kursi kereta tidak mencukupi
Kereta	Entity Class	-	Menyimpan data kereta
ReservasiController	Controller	-	Logika bisnis reservasi
Main	Entry Point	-	Antarmuka menu pengguna

5.2 Penerapan Custom Exception

Program menggunakan tiga buah custom exception yang masing-masing dirancang sesuai konteks domain aplikasi reservasi kereta. DataPenumpangTidakValidException berjenis unchecked (extends RuntimeException) karena merupakan kesalahan pemrograman akibat input yang tidak divalidasi. Sementara itu, RuteTidakDitemukanException dan TiketHabisException berjenis checked (extends Exception) karena merupakan kondisi yang harus diantisipasi dan ditangani secara eksplisit oleh pemanggil.

5.3 Penerapan throw dan throws

Pada method pesanTiket() di class ReservasiController, keyword throw digunakan untuk melempar exception secara eksplisit ketika kondisi tertentu tidak terpenuhi, yaitu: throw new DataPenumpangTidakValidException ketika format NIK salah, throw new RuteTidakDitemukanException ketika kode kereta tidak ada, dan throw new TiketHabisException ketika jumlah tiket yang dipesan melebihi sisa kursi. Keyword throws pada signature method digunakan untuk mendeklarasikan bahwa method dapat melempar RuteTidakDitemukanException dan TiketHabisException.

5.4 Penerapan Multiple Catch dan Finally

Pada class Main, digunakan multiple catch blocks untuk menangani setiap jenis exception secara spesifik. Urutan catch dimulai dari exception yang paling spesifik (InputMismatchException, DataPenumpangTidakValidException, RuteTidakDitemukanException, TiketHabisException) hingga yang paling umum (Exception). Blok finally dimanfaatkan untuk menutup Scanner secara aman ketika pengguna memilih keluar, memastikan resource selalu dibebaskan.

VI. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

- Exception handling dengan blok try-catch-finally memungkinkan program tetap berjalan dengan stabil meskipun terjadi kondisi error yang tidak terduga.
- Custom exception seperti `DataPenumpangTidakValidException`, `RuteTidakDitemukanException`, dan `TiketHabisException` membuat kode lebih ekspresif dan mudah dibaca sesuai konteks domain aplikasi.
- Keyword `throw` digunakan untuk melempar exception secara eksplisit, sedangkan `throws` digunakan untuk mendelegasikan penanganan exception kepada pemanggil method.
- Multiple catch blocks memungkinkan penanganan error yang lebih spesifik untuk setiap jenis exception, sehingga pesan kesalahan yang ditampilkan lebih informatif bagi pengguna.
- Blok `finally` sangat berguna untuk memastikan resource seperti `Scanner` selalu ditutup dengan baik, menghindari kebocoran resource pada program.